Tienda de Ropa

Descripción

Una empresa de ropa requiere el desarrollo de un modelo aprendizaje automático que realice la clasificación de los distintos artículos con base a una imagen e indique a que categoría pertenecen. Las categorías son:

- Camiseta
- Pantalón
- Suéter
- Vestido
- Saco
- Sandalia
- Camisa
- Tenis
- Bolsa
- Botín

Características a tomar en cuenta:

1. La cantidad de **neuronas** que tendrá la capa de **salida** son 10, una por cada tipo de prenda.

2. Ingresar una **imagen** como datos de entrada, determinando la cantidad de **pixeles** que se utilizarán y establecer un valor numérico para cada pixel en un rango donde 0 equivale a un negro total y 255 a un blanco total. Para este ejercicio se utilizarán imágenes de **28 x 28** pixeles (predeterminado por el dataset).

3. Implementar las **capas ocultas**. Se sugiere para este ejercicio la implementación de dos capas ocultas con 50 neuronas(perceptrones) a cada una.

4. Establecer una **función de activación**. La función de activación sugerida es la Función **ReLU**. El estudiante después de resolver este ejercicio quiere implementar otro tipo de función de activación, puede hacerlo.

5. Los **datasets** (imágenes de entrenamiento). En específico se utilizará **fashion_mnist**, que se encuentran en https://www.tensorflow.org/datasets/catalog/fashion_mnist?hl=es-419

   Se debe instalar lo siguiente:

   ```
   pip install tensorflow
   pip install tensorflow-datasets
   ```

   Este catálogo de imágenes fue proporcionado por la empresa Zalando (https://www.zalando.com/) y tiene una investigación acerca de cómo realizó la recopilación de más de 60 mil imágenes de ropa para clasificación.

   Enlaces que puede revisar:
   - https://engineering.zalando.com/tags/zalando-research.html
   - https://github.com/zalandoresearch

Programación

Paso 1. Importar librerías

```
import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import math
import numpy as np
```

Paso 2. Descargar el dataset

```
#Descargar set de datos de Fashion MNIST de Zalando
datos, metadatos = tfds.load('fashion_mnist', as_supervised=True, with_info=True)
```

Escribir a que se refieren los parametros:

as_supervised: _____

_____

_____

with_info: _____

_____

_____

Paso 3. Imprimir los metadatos

```
#Imprimir los metadatos para ver la información el set
metadatos
```

Paso 4. Ordenar cada conjunto de datos en las variables correspondientes

```
#Obtenemos en variables separadas los datos de entrenamiento (60k) y pruebas (10k)
datos_entrenamiento, datos_pruebas = datos['train'], datos['test']
```

Paso 5. Asignar los nombres de las categorías (incluidas en el set) a una variable y mostrarlas.

```
#Asignar los nombres 10 categorias a una variable
nombres_clases = metadatos.features['label'].names
nombres_clases
```

Paso 6. Normalizar los datos.

Acción por la cual, los valores estarán en el rango de 0 a 1, y no de 0 a 255. Para ello se debe dividir cada pixel entre 255, para mejorar el rendimiento del entrenamiento de redes neuronales.

```
def normalizar(imagenes, etiquetas):
  imagenes = tf.cast(imagenes, tf.float32)
  imagenes /= 255 #Aqui lo pasa de 0-255 a 0-1
  return imagenes, etiquetas
```

Luego **todos los datos** de *prueba* y *entrenamiento* utilizarán esa función de normalización.

```
datos_entrenamiento = datos_entrenamiento.map(normalizar)
datos_pruebas = datos_pruebas.map(normalizar)
```

Escribe para que se utiliza la función `map()`

Para que el entrenamiento sea más rápido se recomienda guardar los datos en caché y no utilizar disco, para ello agregar el siguiente código:

```
datos_entrenamiento = datos_entrenamiento.cache()
datos_pruebas = datos_pruebas.cache()
```

Paso 7. Mostrar imfigenes del dataset.

Código para mostrar **una imagen**

```
#Mostrar una imagen de los datos de pruebas
for imagen, etiqueta in datos_entrenamiento.take(1):
  break
imagen = imagen.numpy().reshape((28,28)) #Redimensionar los pixeles

#Dibujar dibujar
plt.figure()
plt.imshow(imagen, cmap=plt.cm.binary)
plt.colorbar()
plt.grid(False)
plt.show()
```

Código para mostrar **una imagen por** cada **categoría** con su etiqueta

```
#Mostrar una imagen por cada categoría con su etiqueta
plt.figure(figsize=(10,10))
for i, (imagen, etiqueta) in enumerate(datos_entrenamiento.take(25)):
  imagen = imagen.numpy().reshape((28,28))
  plt.subplot(5,5,i+1)
  plt.xticks([])
  plt.yticks([])
  plt.grid(False)
  plt.imshow(imagen, cmap=plt.cm.binary)
  plt.xlabel(nombres_clases[etiqueta])
plt.show()
```

Paso 8. Creación de modelo

- Se utiliza una **red de tipo secuencial**.
- Se define una **capa de entrada** de forma manual utilizando la función `Flatten()`, cuya objetivo es, convertir a una dimensión los datos de entrada. Es decir, la matriz original en 2D de 28x28 se convierte a un vector de 784 neuronas.
- Agregar dos **capas ocultas** de 50 neuronas cada una y una función de activación ReLU.
- Agregar la **capa de salida** con 10 neuronas y una función de activación SoftMax, la cuál es muy comúnmente utilizada en redes de clasificación, para que la suma de las neuronas de salidas sólo de cómo resultado 1.

```
#Crear el modelo
modelo = tf.keras.Sequential([
  tf.keras.layers.Flatten(input_shape=(28,28,1)), #1 - blanco y negro
  tf.keras.layers.Dense(50, activation=tf.nn.relu),
  tf.keras.layers.Dense(50, activation=tf.nn.relu),
  tf.keras.layers.Dense(10, activation=tf.nn.softmax) #Para redes de clasificacion
])
```

Paso 9. Compilar el modelo

- Se implementa **`adam`** como optimizador
- La función de perdida utilizada será `SparseCategoricalCrossentropy()`, ya que es común utilizarla en *redes de clasificación.*

```
#Compilar el modelo
modelo.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=['accuracy']
)
```

Paso 10. Dividir por lotes

Se recomienda implementar esta división por la cantidad de datos de entrenamiento (60k) y prueba (10k). Dividir por lotes permite que entrenamientos con gran cantidad de datos se haga de manera más eficiente

Definir una variable para definir el lote TAMANO_LOTE en 32

```
TAMANO_LOTE = 32
```

Define dos variables para tener la cantidad de datos para entrenamiento y prueba.

```
#Los numeros de datos en entrenamiento y pruebas (60k y 10k)
num_ej_entrenamiento = metadatos.splits["train"].num_examples
num_ej_pruebas = metadatos.splits["test"].num_examples
print(num_ej_entrenamiento)
print(num_ej_pruebas)
```

Ajustar las propiedades de los datos de entrenamiento y prueba. La intención es que el entrenamiento se realice de forma aleatoria, debido a que realizará varias veces el recorrido, para ello se implementa la función repeat() y shuffle(). Esto evita que la red aprenda en orden secuencial.

```
datos_entrenamiento = datos_entrenamiento.repeat().shuffle(num_ej_entrenamiento).batch(TAMANO_LOTE)
datos_pruebas = datos_pruebas.batch(TAMANO_LOTE)
```

Paso 11. Entrenar el modelo

```
#Entrenando el modelo
historial = modelo.fit(datos_entrenamiento, epochs=5, steps_per_epoch= math.ceil(num_ej_entrenamiento/TAMANO_LOTE))
```

Paso 12. Mostrar los datos de la función de perdida

```
#Ver la funcion de perdida
plt.xlabel("# Epoca")
plt.ylabel("Magnitud de pérdida")
plt.plot(historial.history["loss"])
```

Paso 13. Realizar predicciones

Imprimir 25 imágenes del set de prueba y realice la predicción de la categoría a la que pertenece cada una de esas imágenes. Identificar con una barra azul si la predicción es correcta o roja si es incorrecta

```
#Crear una cuadricula con varias predicciones, y marcar si fue correcta (azul) o incorrecta (roja)
for imagenes_prueba, etiquetas_prueba in datos_pruebas.take(1):
  imagenes_prueba = imagenes_prueba.numpy()
  etiquetas_prueba = etiquetas_prueba.numpy()

predicciones = modelo.predict(imagenes_prueba)

def graficar_imagen(i, arr_predicciones, etiquetas_reales, imagenes):
  arr_predicciones, etiqueta_real, img = arr_predicciones[i], etiquetas_reales[i], imagenes[i]
  plt.grid(False)
  plt.xticks([])
  plt.yticks([])

  plt.imshow(img[...,0], cmap=plt.cm.binary)

  etiqueta_prediccion = np.argmax(arr_predicciones)
  if etiqueta_prediccion == etiqueta_real:
    color = 'blue'
  else:
    color = 'red'

  plt.xlabel("{} {:2.0f}% ({})".format(nombres_clases[etiqueta_prediccion],
                                100*np.max(arr_predicciones),
                                nombres_clases[etiqueta_real]),
                                color=color)
```

```python

def graficar_valor_arreglo(i, arr_predicciones, etiqueta_real):
  arr_predicciones, etiqueta_real = arr_predicciones[i], etiqueta_real[i]
  plt.grid(False)
  plt.xticks([])
  plt.yticks([])
  grafica = plt.bar(range(10), arr_predicciones, color="#777777")
  plt.ylim([0, 1])
  etiqueta_prediccion = np.argmax(arr_predicciones)

  grafica[etiqueta_prediccion].set_color('red')
  grafica[etiqueta_real].set_color('blue')

filas = 5
columnas = 5
num_imagenes = filas*columnas
plt.figure(figsize=(2*2*columnas, 2*filas))
for i in range(num_imagenes):
  plt.subplot(filas, 2*columnas, 2*i+1)
  graficar_imagen(i, predicciones, etiquetas_prueba, imagenes_prueba)
  plt.subplot(filas, 2*columnas, 2*i+2)
  graficar_valor_arreglo(i, predicciones, etiquetas_prueba)
```

Realizar la predicción de una imagen

```python
#Realizar la predicción de una imagen
imagen = imagenes_prueba[24]
imagen = np.array([imagen])
prediccion = modelo.predict(imagen)

print("Prediccion: " + nombres_clases[np.argmax(prediccion[0])])
```

Actividades:

1. Responder las preguntas que se encuentran dentro de la guía.
2. Modificar el código para que un usuario pueda cargar una imagen y el modelo pueda clasificarlo. Hacerlo interactivo
3. Investigar ¿qué es el formato h5 y para que sirve?
4. Exporte el modelo al formato h5.